

4.8 Machine Learning Methodology for Property Value Estimator

4.8.1 Overview

The Property Value Estimator is a machine learning (ML) subsystem embedded within the e-Rumah platform. Its primary function is to generate automated property valuation estimates for residential properties submitted through the platform, supporting homeowners and administrative staff in assessing market value without requiring a full manual appraisal. The estimator processes structured property attributes to produce a predicted market price, which serves as a reference figure for downstream processes such as eligibility screening, loan structuring, and offer comparison.

This section describes the machine learning methodology adopted for the estimator, covering problem framing, dataset considerations, feature engineering, algorithm selection, model training and evaluation strategy, and integration with the broader system architecture.

4.8.2 Problem Framing

Property valuation is formulated as a supervised regression problem. Given a set of input features describing a property, the model learns a mapping function $f(X) \rightarrow y$, where X represents the feature vector of property attributes and y is the continuous target variable representing the estimated market price in Malaysian Ringgit (MYR).

This approach is consistent with the Automated Valuation Model (AVM) paradigm widely adopted in real estate analytics (Hayes, 2026b). Unlike classification tasks that produce discrete labels, regression produces a continuous numerical output, which is appropriate for price prediction where fine-grained estimates are more useful than broad categories.

4.8.3 Dataset and Data Sources

The model is trained using a dataset of residential property transaction records. The primary intended source is the National Property Information Centre (NAPIC), maintained by the Valuation and Property Services Department (JPPH) under the Ministry of Finance Malaysia. NAPIC publishes transaction data covering residential property sales across all states in Malaysia, including property type, location, floor area, and transacted price.

As a supplementary or alternative source, publicly available property listing data from Malaysian real estate platforms may be used, subject to data availability and ethical considerations. In the context of system development and testing, a representative sample dataset is used to train and validate the model before deployment.

Pre-processing steps applied to the raw dataset include:

- Removal of duplicate and incomplete records

- Filtering of outlier transactions that fall outside statistically reasonable price ranges
- Normalisation of categorical variables using consistent encoding schemes
- Imputation of missing numerical values using median substitution
- Stratified splitting of data into training (70%), validation (15%), and test (15%) subsets

4.8.4 Feature Engineering

Feature engineering involves transforming raw property data into structured numerical inputs that the model can process effectively. The features selected for the estimator are drawn from domain knowledge of Malaysian residential property valuation and are categorised as follows.

Table 4.20: Input Features for the Property Value Estimator

Category	Feature	Description
Property Type	property_type	Residential category: terrace, semi-detached, condominium, bungalow, flat
Size	built_up_area_sqft	Interior floor area in square feet
Size	land_area_sqft	Total land lot size (applicable for landed properties)
Location	state	Malaysian state where property is located
Location	district	District or municipality within the state
Location	postcode	Postal code used as a granular location proxy

Property Age	age_years	Number of years since construction completion
Legal Status	tenure_type	Freehold or leasehold; remaining lease years if leasehold
Interior	num_bedrooms	Number of bedrooms
Interior	num_bathrooms	Number of bathrooms
Condition	furnishing_status	Unfurnished, partially furnished, or fully furnished
Transaction	transaction_year	Year of recorded transaction for temporal adjustment

Categorical features such as `property_type`, `state`, `district`, and `tenure_type` are encoded using target encoding or label encoding depending on cardinality. High-cardinality fields such as `postcode` are encoded using frequency-based or mean-target encoding to avoid dimensionality explosion.

4.8.5 Algorithm Selection

A comparative evaluation of three regression algorithms is conducted to identify the most suitable model for this use case. The algorithms considered are Multiple Linear Regression (MLR), Random Forest Regression, and Extreme Gradient Boosting (XGBoost). Each algorithm represents a different level of model complexity and interpretability, and their relative strengths and limitations in property valuation contexts are summarised in Table 4.21.

Table 4.21: Comparison of Candidate Regression Algorithms

Algorithm	Complexity	Interpretability	Strengths	Limitations
Multiple Linear Regression (MLR)	Low	High	Simple, fast, transparent coefficients	Assumes linearity; poor with complex feature interactions
Random Forest	Medium	Medium	Robust to noise and overfitting; handles non-linearity	Slower inference; less accurate than boosting on tabular data
XGBoost	High	Medium	High accuracy; handles missing values; feature importance scores	Requires hyperparameter tuning; less transparent than MLR

Multiple Linear Regression (Hayes, 2026a) is included as the baseline model. Despite its simplicity, it provides a transparent benchmark against which more complex models can be measured. Its coefficient outputs also carry interpretive value in explaining the directional influence of each feature on the predicted price.

Random Forest Regression is an ensemble method that constructs multiple decision trees over bootstrapped subsamples of the training data and averages their predictions (GeeksforGeeks, 2026). It is robust against overfitting and captures non-linear relationships between features and price without requiring extensive feature engineering.

XGBoost (Extreme Gradient Boosting) is selected as the primary model for deployment based on its consistently superior performance on structured tabular datasets, as evidenced in existing literature on AVM systems (Chen & Guestrin, 2016). XGBoost builds an ensemble of shallow trees in a sequential, gradient-descent manner, correcting the residual errors of prior trees at each stage. It natively handles missing values and provides SHAP (SHapley Additive exPlanations) compatibility for post-hoc interpretability, which is important for stakeholder trust in a financial context.

4.8.6 Model Training and Hyperparameter Tuning

All three models are trained using the training subset of the processed dataset. For XGBoost, hyperparameter optimisation is performed using k-fold cross-validation ($k = 5$) on the training set to reduce variance in performance estimation. The hyperparameters subject to tuning include:

- `n_estimators`: number of boosting rounds (range: 100 to 1000)
- `max_depth`: maximum tree depth (range: 3 to 10)
- `learning_rate` (eta): step size shrinkage (range: 0.01 to 0.3)

Hyperparameter search is conducted using grid search or randomised search, with the best configuration selected based on the lowest mean cross-validated Root Mean Squared Error (RMSE) on the training set. Early stopping is applied during training to prevent overfitting when the validation RMSE fails to improve over a defined number of consecutive rounds.

4.8.7 Model Evaluation and Performance Metrics

Model performance is assessed on the held-out test set using three standard regression metrics:

Table 4.22: Evaluation Metrics for Model Performance Assessment

Metric	Formula	Interpretation
Mean Absolute Error (MAE)	$MAE = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	Average absolute difference between predicted and actual price
Root Mean Squared Error (RMSE)	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$	Penalises large prediction errors; sensitive to outliers
Coefficient of Determination (R^2)	$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$	Proportion of price variance explained by the model; higher is better (max 1.0)

An acceptable model performance threshold is defined as an R^2 value of 0.80 or above on the test set, indicating that at least 80% of the variance in property prices is explained by the model. This threshold is aligned with AVM accuracy standards commonly cited in valuation practice literature (Chalvatzis & Hristu-Varsakelis, 2020). The selected model is also evaluated for consistency across property types and geographic regions to identify potential biases in prediction.

4.8.8 Model Interpretability

Interpretability is a key requirement for the estimator given the financial implications of property valuation outputs. Two complementary interpretability mechanisms are incorporated:

- **Feature Importance Scores:** XGBoost natively computes feature importance based on the frequency and gain associated with each feature across all trees. These scores are displayed in the system's administrative dashboard to allow operators to understand which factors most influence predicted values.
- **SHAP (SHapley Additive exPlanations):** SHAP values are computed for individual predictions to provide localised explanations of why a specific property received a particular estimate. This is surfaced to end users in the form of a simplified

explanation panel, for example indicating that the property's location and built-up area are the two largest contributors to its estimated value.

4.8.9 Integration with the e-Rumah Platform

The trained ML model is serialised and deployed as a backend service accessible via a RESTful API endpoint. When a user or agent submits property details through the Application Management module, the frontend sends a structured JSON payload to the estimation endpoint. The backend loads the serialised model, applies the same pre-processing pipeline used during training, and returns the predicted market value along with a confidence interval and feature contribution summary.

The integration architecture ensures that the model is decoupled from the main application logic, allowing the model to be retrained and updated independently without disrupting the frontend. Model versioning is maintained to enable rollback if a newer model degrades in production performance.

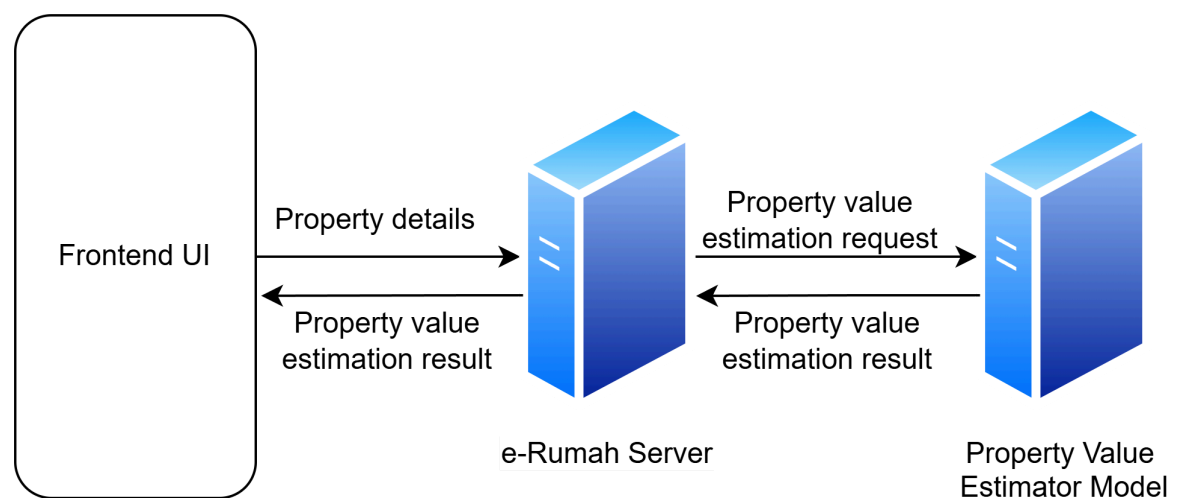


Figure 4.17 Data flow between the user interface, the e-Rumah server and the Property Value Estimator Model

4.9 Chapter Summary and Evaluation

This chapter has produced a comprehensive system design for e-Rumah across all five modules. The architecture section documented the hybrid Client-Server, Broker and MVC pattern, explaining how each component contributes to maintainability and interoperability. The process design presented the context diagram, data flow diagram, use case diagrams and activity diagrams covering UC001, UC002, UC003 and UC006.

The data design introduced the entity relationship diagram covering all modules. The UI design section articulated five elderly-centric design principles and provided wireframes for five key screens, supplemented by user flow description. Security was specified through a defence-in-depth model covering authentication, role-based access control, encrypted communication, signed file access, input validation and a write-only audit log. Three algorithms covering due-date classification, upload validation and flag routing were specified through pseudocode. The machine learning methodology for the Property Value Estimator subsystem is also explained and the selection of the XGBoost algorithm is justified. Chapter 5 will present the implementation of these designs.